Comp 533 - Assignment 2: Multi-Threaded Map-Reduce

Date Assigned: Thu, Jan 30, 2025

Early Completion Date: Tue, February 11, 2024, 11:55 pm (+5% extra credit)

Completion Date: Fri, February 14, 2025, 11:55 pm

Late Date: Fri, February 21, 2025 (-10% late penalty)

Scary Late Date: Fri, February 28, 2025 (-30 % late penalty)

Assignment goals:

- Get practice in forking threads and synchronizing them using a bounded buffer (blocking queue).
- Get practice in dispatching multiple rounds of work to worker threads.
- Get practice in generating and handling interrupts and multi-thread user-controlled termination.
- Get implementation-level understanding of thread Join and Barrier abstractions, and Hadoop's Partition abstractions.
- Get practice in using the implemented abstractions to implement a real problem.

Topic	Learning Resources
Java APIs	Wait and Notify java.util.concurrent.BlockingQueue java.util.concurrent.ArrayBlockingQueue
Map Reduce	Map Reduce Tutorial
Bounded Buffer	Wikipedia
Master/Slave Technology	Wikipedia
Barrier synchronization	Wikipedia
Fork-Join synchronization	Wikipedia

The challenge in this assignment is to concurrently reduce a set of Key-Value items in multiple threads. As in A1, the main thread does the mapping. As in the MapReduce framework, this reduction occurs in two steps.

1. Input is mapped concurrently to Key-Value pairs and *partially reduced*.
    a. This list is broken into sub-lists that are concurrently reduced. This means that a given key may have values spread across multiple lists.
    b. These lists are then partitioned into lists with non-overlapping keys. Now, all the values of a given key should be contained in one partition.
2. Key-Value pairs are fully reduced concurrently to print the final result.
    a. The independently partitioned lists are concurrently further reduced (by the same threads) that did partial reduction into a set of independent reductions.
    b. These independent reductions are then serially combined into one final result.

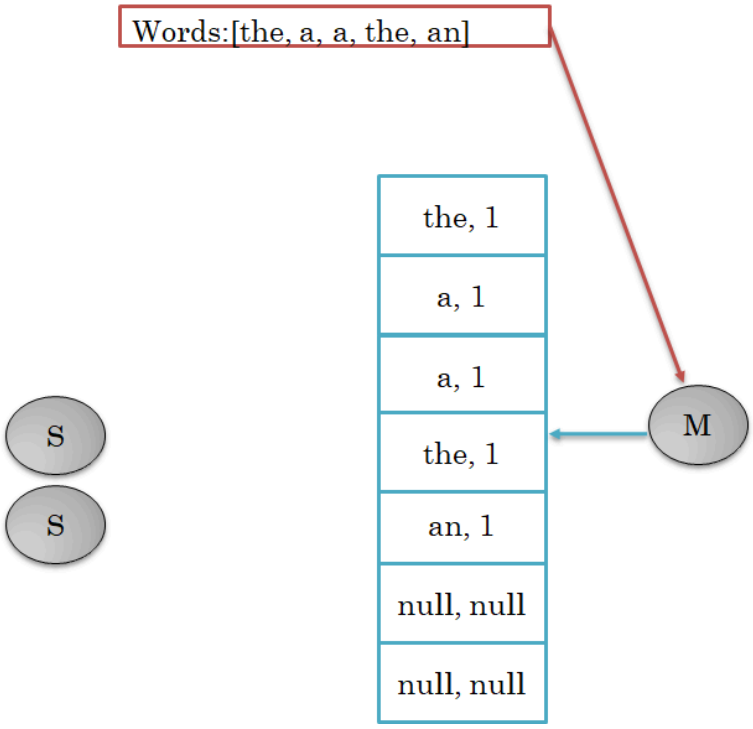
How work is divided into concurrent threads is different from the MapReduce framework, which exploits both concurrency and distribution and does not use many of the concepts to be learned in this assignment such as bounded buffers.

Graphically Illustrated Algorithm using Token Counting

Let us say that the input is the token sequence to be counted is: the, a, a, the, an

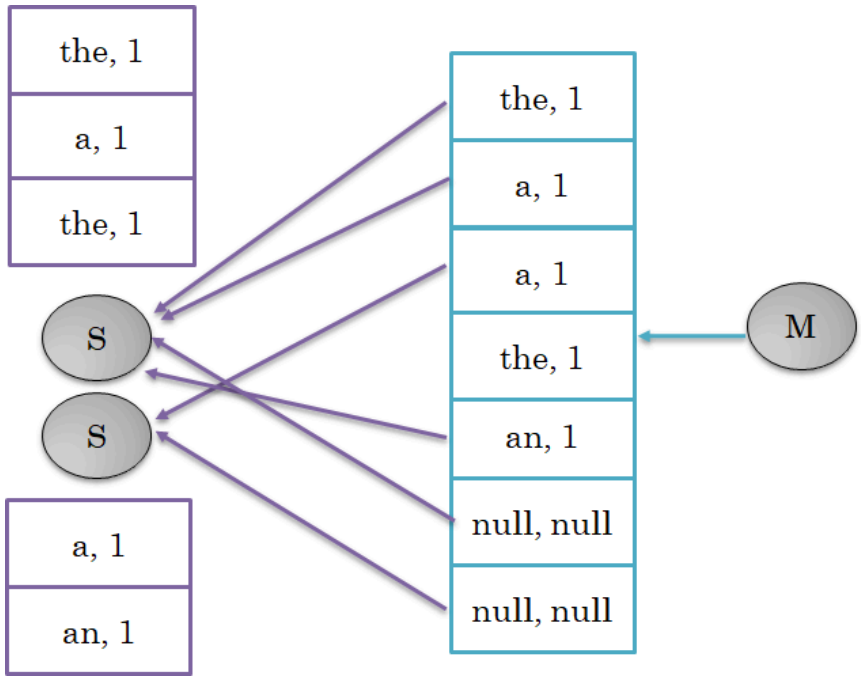
As in A1, the main/master thread maps it into Key-Value pairs. The reduction step will be taken concurrently by helper/slave/worker threads. This means the main thread must put the mapped objects into some shared data structure accessed by the helper threads. This data structure is a concurrently accessed ordered list of items of bounded length called a bounded buffer. The main thread produces or puts items into the buffer and the helper threads consume or take them from the buffer.

We see below the items produced by the main thread after mapping the input tokens.



The main thread knows there are two helper threads, so it puts a special end of input item for each thread. Once a thread encounters such an item, it stops consuming.

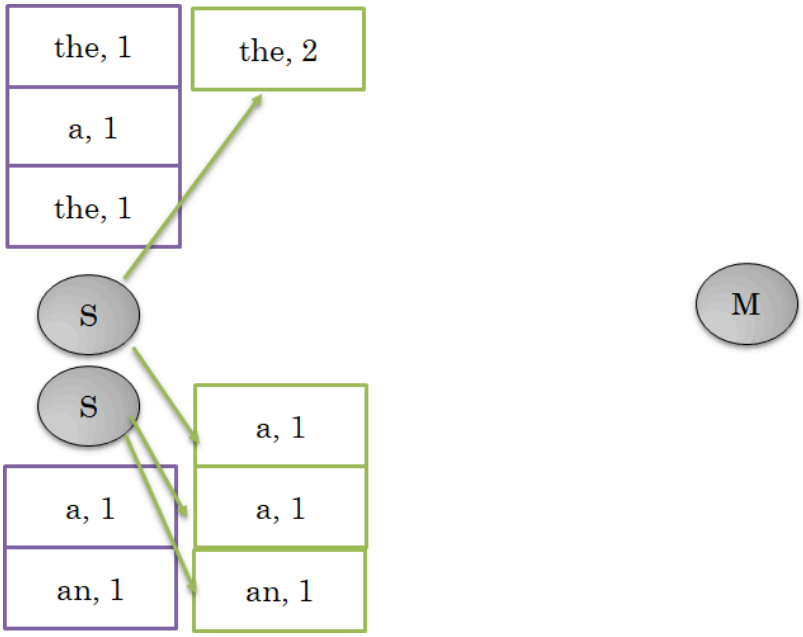
Which items are consumed by a particular helper thread is nondeterministic. The following figure shows one possible consumption sequence.



The top helper thread has consumed items 1, 2, 4, and 6 and the bottom helper thread has consumed items 3,5, and 7. Each thread stops after consuming an end of input item. All non-terminating consumed items are put in a *mapped list* (purple boxes) not shared with other threads.

As we see in this example, the two private lists have items with an overlapping key: a. This means that the reductions carried out by the two helper threads will not be final. These partial reductions will have to be further reduced.

The result of the partial reductions performed by each helper thread do not however go into private lists. Instead, they go into shared *partially-reduced lists*, one owned by each helper thread, to which any helper thread can write, as shown below:

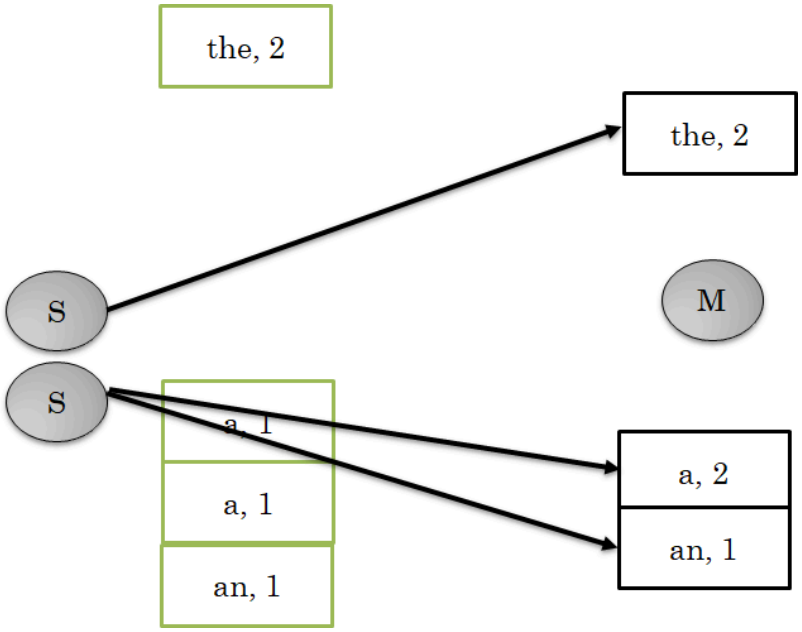


Here the top helper thread computes two reduced items: (the, 2) and (a, 1). It puts the first item into the shared list (green boxes) it owns and the second one in the shared list the other helper thread owns. The bottom helper thread computes two items: (a, 1) and (an, 1). It puts both into the shared list it owns.

What is the criteria for determining the destination of a reduced item? The set of possible keys are partitioned among all helper threads by a partitioning algorithm implemented by a partitioning object implemented by you. Here the portioning algorithm assigned the top helper thread the key, **the**, and the bottom one the keys, **a** and **an**. The two shared lists are thus guaranteed to have non-overlapping keys.

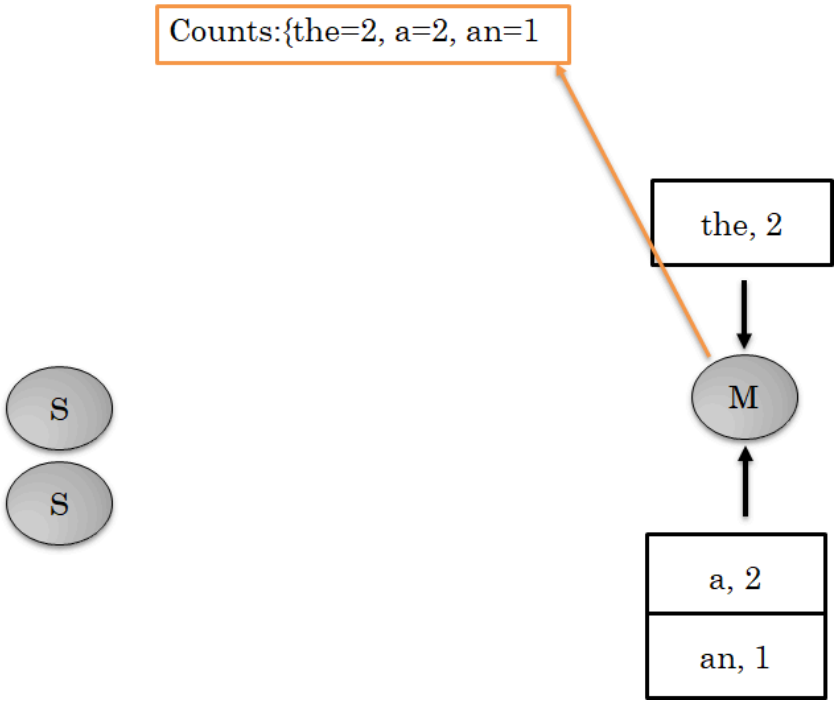
The task of each helper thread is to now perform final reductions on the partially-reduced list it owns. However, a helper thread cannot perform this step until all threads have performed their partial reductions. The threads use a **barrier synchronization object** to wait for this event. You will be responsible for implementing this object using the thread concepts you have learned.

The result of the final reductions performed by a helper thread on its partially reduced list goes into its final reduction shared list, which it owns, as shown below:



These per-thread final lists will be read by the master thread when all of these lists have been computed. To wait for this event, the master and helper threads use a joiner synchronization object to be implemented by you.

Once this event occurs, the master thread simply merges the results of each full reduced list into a map to produce the final output.



This process repeats for each line (round) of input until the user quits. This means the main thread waits for the next line of input and each slave thread waits for the main thread to indicate that the line has been read.

Here is how your implementation will change: A master thread will perform many of the steps it executed in A1 including processing input tokens, mapping these tokens to Key-Value items, and announcing and printing the final results. These steps will be partitioned into the model, view, and controller as in A1 – so the implementation of these tasks do not change. The master thread will no longer perform the reduction step. Instead, it will take several additional steps: It will prompt the user for number of helper slave threads to be created, create the slave threads, add Key-Value items to a bounded buffer, wait for the final result using a new joiner synchronization abstraction, and combine different partitioned lists into a final result. The first input task will be done in a modified or extended controller, the remaining will be done in a modified or extended model.

A slave object will execute the steps of the slave threads which include consuming items in the Key-Value bounded buffer, performing partial reductions, partitioning them, waiting for its partition to be complete using a barrier synchronization abstraction, computing final reductions of its partition, and using the join abstraction to signal to the master thread that its final reduction is complete.

As before multiple, independent lines of user input will be processed. A single bounded buffer will be used for all lines. That is, there is one bound buffer that the master thread adds input items into, and threads initially get items out of. This buffer should be reused each round.

The master thread adds (and slave threads must process) special end of input items in the bounded buffer to separate the Key-Value items computed from different input lines.

The joiner synchronization object allows a single thread to wait for a set of threads to complete some action and the barrier abstraction allows a set of threads to wait for all members of the set to complete some action. You will implement both abstractions.

Two new partitioner objects will determine how Key-Value parts are separated.

The configuration must be changed to indicate all aspects required by A2.

It will be best to copy your A1 project into a new one. You can directly modify rather than use inheritance to extend the A1 classes.

New Configuration Methods

For localchecks, you need to edit the stubs of the following methods to return non null values:

```
Class getPartitionerClass();
Object getPartitioner();
Class getJoinerClass();
Class getBarrierClass();
Class getSlaveClass();

// return some instance of the Barrier and Joiner classes
// in the method below as these are not singletons
    Object getBarrier(int aNumThreads);
```

```
Object getJoiner(int aNumThreads);
```

Slave Class

Create a slave class that implements a slave interface that extends Java Runnable. This object serves two purposes. First, it implements the run() method. This method is invoked by the slave thread. This is why it is called the slave class.

The method also serves as an object to coordinate the associated slave and the master thread. The slave thread calls a wait() on its slave object to wait for the next input line. The master thread calls notify() on the slave object of a slave thread to unblock it when the next line is available. As you know, a thread cannot notify() another thread directly. Thus, we are essentially notifying a slave thread by calling notify() on its slave object.

As the notify() method is not public, the runnable slave class implements not only the run() method but also a public **synchronized** *slave notifying method* that invokes notify() on the runnable slave object (this) each time a new input line is read.

Before any notify() in any class be sure to call the appropriate trace method:

```
/**
 * To be called before doing a notify on this
 */
protected void traceNotify () {
      trace(NOTIFY);
}
```

Implement the slave toString() method to use the SLAVE constant in the tracer class:

```
      public static String SLAVE = "Slave"; // value returned by toString()
method of a slave
```

Implement a stub run() method for now. You will incrementally add and modify methods in the interface and class based on the directions below.

Return the class object for this class in the getSlaveClass() method of the configuration.

Specifying and Processing Number of Threads

Add to the model the following additional property:

NumThreads:int (Editable) // number of slave/helper threads created by the model

In addition, add the following readonly property:

List<Thread> threads (Readonly)// the list of slave threads created by the model.

The NumThreads setter creates the required number of slave threads. It passes to the constructor of each slave runnable class a unique number in the range 0..Num Threads - 1, which will be called the slave number of the thread that executes this runnable. It passes to the slave constructor the model (this). It sets the name of the thread to be "Slave<SlaveNumber>".

When setting the number of the threads you should fire a property change event with the old and new number of threads. The name should be "NumThreads"

Before any round of input is processed, an extended controller now reads from the user the number of threads and calls the model setter above, which announces the change to this property to the view, which in turn prints the event as before:

// The controller prompts for all input – first for #threads

```
Thread[main,5,main]:Controller:Please enter the number of threads
```

// The user indicates how many slave threads are created.

```
3
```

// The view prints the notification from the model announcing the changed NumThreads property

```
Thread[main,5,main]:View:java.beans.PropertyChangeEvent[propertyName=NumT
hreads; oldValue=0; newValue=3; propagationId=null; source=Model]
```

// The model creates three new threads, adds and the view prints the notification from the model announcing the changed Threads property storing a list of created threads

```
Thread[main,5,main]:View:java.beans.PropertyChangeEvent[propertyName=Thre
ads; oldValue=null; newValue=[Thread[Slave0,5,main],
Thread[Slave1,5,main], Thread[Slave2,5,main]]; propagationId=null;
source=Model]
```

// As in A1, multiple input lines are processed and printed:

```
Thread[main,5,main]:Controller:Please enter quit or a line of tokens
separated by spaces
```

// The user provides a line of input

```
orange apple orange mango lychee lychee soursop lychee orange mango
lychee
```

…
…

Partitioner and Partitioner Factory

Define a partitioned interface that provides an instance method, called getPartition(). This method takes as an argument a key, a value, and the number of partitions (in that order). The types of key and value of the method should be generic though it will be called in this assignment with String and Integer values

```
public int getPartition(K key,V value, int numberOfPartitions);
```

Implement a partitioner class (inspired by the Hadoop Partitioner interface) that implements the interface. This class should implement a version of the partitioner interface in which the key type parameter is bound to String (while the value type parameter is generic for that class.) Its implementation of the getPartition method returns a value in the range 0 .. number of partitions - 1 indicating the partition to which the key and value belong.

It does the partitioning based on the first character of the String key. If the first character of the key is not a letter (which can be determined by Character.isLetter()), it returns the 0th partition. Otherwise it converts the character to lowercase using the

Character.toLowerCase method. Based on the offset of the lowercase character from ‘a’ (which can be determined by subtracting ‘a’ from it) it assigns the appropriate partition using the following partition balancing constraints, which assign to each partition a series of consecutive letters.

Let p be the number of partitions. Let max partition size, s, be the ceiling of the number of letters divided by the number of partitions, that is, ceiling ((‘z’ - ‘a’ + 1) / (double) p). Then the partition assigned to a key whose first character is a letter with the lowercase value, f, is: floor ((f - ‘a’ + 1)/(double) s). The Math.floor() and Math.ceiling() operations can be used for floor and ceiling.

Implement a partitioner factory class that provides an editable static property called Partitioner whose type is the partitioned interface above. This means you need a static getter and setter for the property. The default value of this property is a singleton instance of the partitioner class.

Since the factory provides static methods, it cannot be generic. Thus, the type of the Partition property of the factory class should be the generic interface in which the key and value types are bound to String and Integer respectively.

Implement the partitioner toString() method to use the PARTITIONER constant in the tracer class:

```
	public static String PARTITIONER = "Partitioner"; // value returned by
toString() method of a partitioner
```

Call the following trace method:

```
	/*
	 * Called before returning from the getPartition() method in the
partitioner
	 */
	protected void tracePartitionAssigned(Object aKey, Object aValue, int
aPartitionNum, int aNumPartitions ) {
		trace (PARTITION_ASSIGNED + aKey + ":" + aValue + ":" +
aPartitionNum + ":" + aNumPartitions );
	}
```

Change the aspects of the configuration having to do with the partitioning interface, factory, class, and default object:

```
	// The partitioner object retrieved from the partitioner factory.
	public Partitioner getPartitioner();
	// The partitioner factory
	public Class getPartitionerFactory();
	//The partitioner class
	public Class getPartitionerClass();
```

At this point, the local checks suite testing the partition factory and objects should work.

Barrier

The role of a barrier is to allow each thread in some team to wait until all members of the team have finished some task.

Implement a barrier class and interface that provides the following constraints:

1. Define a constructor taking an int parameter indicating the number of threads that will call the barrier() method described below. Let us call this the (barrier) count.
2. Define a parameterless method, called *barrier*, to be invoked on a Barrier instance in multiple phases by multiple threads. Each call to this method should block until a count number of calls to this method by different threads have been made. This means count threads will be unblocked at once.
3. Use wait() to block and notifyAll() to unblock.
4. After each call to notifyAll(), the state of the barrier should be its initial state - the state after the constructor is called. This allows the threads to use the same barrier to synchronize multiple times.

Implement the barrier toString() method to use the BARRIER constant in the tracer class:

```
	public static String BARRIER = "Barrier"; // value returned by
toString() method of a barrier
```

Call the following methods to trace the above steps:

```
/**
	 * To be called in the model after a barrier has been created
	 */
	protected void traceBarrierCreated(Object aBarrier, int aNumThreads) {
		trace (BARRIER_CREATED  + aBarrier + ":" + aNumThreads);
	}
	/**
	 * To be called in the the barrier before a wait()
	 */
	protected void traceBarrierWaitStart(Object aBarrier, int aNumThreads,
int aNumWaitingThreads) {
		trace (BARRIER_WAIT_START + aBarrier + ":" + aNumThreads + ":" +
aNumWaitingThreads);
	}
	/**
	 * To be called in the barrier after a wait()
	 */
	protected void traceBarrierWaitEnd(Object aBarrier, int aNumThreads, int
aNumWaitingThreads) {
		trace (BARRIER_WAIT_END + aBarrier + ":" + aNumThreads + ":" +
aNumWaitingThreads);
	}
	/**
	 * To be called in the the barrier after all blocked threads are
unblocked
	 */
```

```
	protected void traceBarrierReleaseAll(Object aBarrier, int aNumThreads,
int aNumWaitingThreads) {
		trace (BARRIER_RELEASE_ALL + aBarrier + ":" + aNumThreads + ":" +
aNumWaitingThreads);
	}
```

Change barrier aspects of the configuration. In particular, implement the methods:

```
// return an instance of the Barrier class:
Object getBarrier(int aNumThreads);
Class getBarrierClass();
```

At this point, the local checks suite testing the barrier object should work.

Joiner

The role of a joiner is to allow a thread to wait until all threads it has forked have finished some task.

Implement a Joiner class and interface with the following constraints:

1. It has a constructor taking an int parameter indicating the number of threads that will call the finished method described below. Let us call this the joiner count.
2. It has a parameterless instance method, called, *finished*, to be invoked in multiple phases by multiple finishing threads.
3. It also has a parameterless instance method, called, *join*, also to be invoked in multiple phases by multiple finishing threads.
4. join() blocks until count finished() calls have been made.(This is unlike the Thread join() method which waits until a thread terminates)
5. It uses wait() to block and notify() to unblock.
6. After each call to notify, the state of the joiner should be its initial state – the state after the constructor is called. This allows threads to use the same joiner to synchronize multiple threads to zero to start the next phase.

Implement the joiner toString() method to use the JOINER constant in the tracer class:

```
	public static String JOINER = "Joiner"; // value returned by toString()
method of a joiner
```

Call the following tracing methods to trace the corresponding actions:

```
protected void traceJoinerCreated(Object aJoiner, int aNumThreads) {
		trace (JOINER_CREATED +aJoiner + ":" + aNumThreads);
	}
	protected void traceJoinerWaitStart(Object aJoiner, int aNumThreads, int
aNumFinishedThreads) {
		trace (JOINER_WAIT_START + aJoiner + ":" + aNumThreads + ":" +
aNumFinishedThreads);
```

```
        }
        protected void traceJoinerWaitEnd(Object aJoiner, int aNumThreads, int
aNumFinishedThreads) {
                trace (JOINER_WAIT_END + aJoiner + ":" + aNumThreads + ":" +
aNumFinishedThreads);
        }
        /*
         * Call at the start of the finish method
         */
        protected void traceJoinerFinishedTask(Object aJoiner, int aNumThreads,
int aNumFinishedThreads) {
                trace (JOINER_FINISHED_TASK + aJoiner + ":" + aNumThreads + ":" +
aNumFinishedThreads);
        }
        /*
         * Call after the blocked forker is unblocked
         */
        protected void traceJoinerRelease(Object aJoiner, int aNumThreads, int
aNumFinishedThreads) {
                trace (JOINER_RELEASE + aJoiner + ":" + aNumThreads + ":" +
aNumFinishedThreads);
        }
```

Change joiner aspects of the configuration. In particular, implement the methods:

// return an instance of the Joiner class:

```
Object getJoiner(int aNumThreads);
```

// return the Joiner class:

```
Class getJoinerClass();
```

At this point, the local checks suite testing the joiner object should work.

New and Extended Model Properties

Add now the following readonly properties to the model.

KeyValueQueue: This is a bounded buffer. The size of the buffer is a named constant in the tracing class (BUFFER_SIZE) and its type is BlockingQueue. Use the ArrayBlockingQueue implementation of BlockingQueue to implement this property. The items of the queue are Key-Value instances. A single queue is used for all rounds of input.

ReductionQueueList: This type of the property is a Java List. Use the ArrayList implementation of List in your implementation. Its elements are instances of Java's LinkedList, which is essentially a non-blocking queue. We will refer to each LinkedList element of this list as a reduction queue. Like the bounded buffer above, a reduction queue will collect Key-Value instances. A reduction queue is

the list to which partially reduced key values are added by each slave. In the partial reduction phase, slaves add Key-Value items to the queue. In the final reduction phase, each reduction queue is processed by the slave that owns it.

Joiner: An instance of the Joiner interface used for processing of all input. This instance will be different from one returned by the configuration. All joiner instances, of course, have the same class. As this class is not a singleton, we will have multiple instances of this class.

Barrier: An instance of the Barrier class used for processing of all input: Agan, this instance will be different from one returned by the configuration

Extend the NumThreads setter to add a separate reduction queue to the ReductionQueueList for each slave thread.

Thread Safe Reduction Queue

Multiple slaves can concurrently add key-value items to a reduction queue based on the partition associated with the queue. You can use blocks synchronized to the same queue (synchronizing object) to ensure atomic additions. The syntax is:

synchronized (<synchronizing object>) { … }

This statement is equivalent to executing a synchronized method in the synchronizing object whose body is the block of code within curly braces.

Master Thread Tasks

As part of the master thread, the setter of InputString method now performs the following steps, some of which it already does:

1. *Initialization of next round of token processing*: It initializes all data structures associated with processing of the tokens input in this round. This includes clearing the final Result, clearing the reduction queues (which should be empty).
2. *Possible unblocking of slave threads*: It invokes the slave notifying method of the Runnable of each thread. If the thread is waiting on this Runnable, then it will unblock.
3. *Problem split*: It uses the String split() function to tokenize the string. It converts each token to an instance of the Key-Value object using the map() method (of the mapper returned by the mapper factory).
4. *Bounded buffer production*: It adds this object to the bounded buffer using the blocking put() operation, which will be consumed by the slaves. This means the master and slave threads may interleave their execution. Be sure to call the trace methods below.
5. *End of input enqueue*: After all items have been enqueued, it indicates the end of input to each slave by adding a Key-Value object with the components (Null, Null) for each slave. Thus, if there are 3 slaves, three such objects will be added to the buffer. (What would go wrong in this algorithm if only one end of the input object was added for all slaves rather than N such objects for N slaves?)

6. *Join*: It invokes the join() method in the joiner stored in the Joiner property to wait for the result to be computed by all slaves. (Why can we not call the join() method on each of the slaves/worker threads?)
7. Merge partitioned results serially: It serially combines into the Result property the slaves' final reductions of their partitions in the reduction queue list.
8. It announces a change to the Result property to its observers. The announced old value should be null and the new property should be the Result Map.

The following trace methods should be used by the master thread when it manipulates the bounded buffer:

```
/**
 * To be called by a master thread before adding an element to
 * a queue
 */
protected void traceEnqueueRequest(Object anElement) {
	trace(ENQUEUE + anElement);
}
/**
 * To be called by a master thread after adding an object to
 * a queue
 */
protected void traceEnqueue(Object aQueue) {
	trace(ENQUEUE + aQueue);
}
```

The following trace method should be used by the master thread when it combines the slaves' final reduction:

```
/**
 * Call after adding reduced map to final map
 */
protected void traceAddedToMap(Object anOriginalMap, Object aReducedMap)
{
	trace( ADDED_TO_FINAL_MAP + anOriginalMap + ":" + aReducedMap);
}
```

The new steps, like the previous ones, are traced using the relevant tracing code.

The following partial trace shows these master steps, showing names I chose for the various classes, which you do not have to adhere to:

// The master calls notify on each slave runnable, but no slave waits on its runnable in the first round

```
Thread[main,5,main]:Slave:notify
Thread[main,5,main]:Slave:notify
Thread[main,5,main]:Slave:notify
```

// The mapper converts a string to a (key, value) pair, which in this case is an input token and its count of 1

```
Thread[main,5,main]:Token Counting Mapper:Map:orange:(orange,1)
```

// The master requests an enqueue of the (key, value) pair in the bounded buffer,

```
Thread[main,5,main]:Model:enqueue:(orange,1)
```
// The enqueue returns without blocking, with the trace showing the new queue contents
```
Thread[main,5,main]:Model:enqueue:[(orange,1)]
```
// One more item added without blocking
```
Thread[main,5,main]:Token Counting Mapper:Map:apple:(apple,1)

Thread[main,5,main]:Model:enqueue:(apple,1)
Thread[main,5,main]:Model:enqueue:[(apple,1)]
```
// The next operation can block as the bounded buffer is of size 2. Why "can"?
```
Thread[main,5,main]:Model:enqueue:(orange,1)
```
…
// The last enqueue master request and its execution
```
Thread[main,5,main]:Model:enqueue:(lychee,1)
Thread[main,5,main]:Model:enqueue:[(lychee,1)]
```
// The master now requests enqueue of one terminating (null, null) item for each slave, three in all
```
Thread[main,5,main]:Model:enqueue:(null,null)
```
…
```
Thread[main,5,main]:Model:enqueue:(null,null)
```
…
```
Thread[main,5,main]:Model:enqueue:(null,null)
```
// After enqueuing end of input tokens, the master thread waits on the joiner for slaves to finish their work
```
Thread[main,5,main]:Joiner:Joiner Wait Start:Joiner:3:0

```
…
// The last slave to finish does a notify() on the Joiner (see below)s
…

// The master thread unblocks:
```
Thread[main,5,main]:Joiner:Joiner Wait End:Joiner:3:0
```
// The master threads adds the reduction queues data into the final map
```
Thread[main,5,main]:Model:Added to Final Map:{apple=1}:[(apple,1)]
Thread[main,5,main]:Model:Added to Final Map:{orange=3, apple=1, lychee=4,
mango=2}:[(orange,3), (lychee,4), (mango,2)]
Thread[main,5,main]:Model:Added to Final Map:{orange=3, apple=1, lychee=4,
soursop=1, mango=2}:[(soursop,1)]
```
// The master thread announces the change in the model and then prints the view response:
```
Thread[main,5,main]:View:java.beans.PropertyChangeEvent[propertyName=Result;
oldValue=null; newValue={orange=3, lychee=4, apple=1, soursop=1, mango=2};
propagationId=null; source=Model]
```

// Controller code to prompt the user for the next line

```
Thread[main,5,main]:Controller:Please enter quit or a line of tokens separated
by spaces
cherry blueberry
```

```
Thread[main,5,main]:View:java.beans.PropertyChangeEvent[propertyName=InputStri
ng; oldValue=orange apple orange mango lychee lychee soursop lychee orange
mango lychee; newValue=cherry blueberry; propagationId=null; source=Model]
```
// Again, the master thread calls the notify method one each slave's runnable, this time the slaves are waiting on them and they are all unblocked
```
Thread[main,5,main]:Slave:notify
Thread[main,5,main]:Slave:notify
Thread[main,5,main]:Slave:notify
```
…

Slave Thread Tasks

Each slave thread now executes a loop in the run() method of the slave runnable class.

This loop invokes code in both this Runnable object, and the model object. In each iteration of the loop, the slave performs three kinds of tasks:

1. Splitting bounded buffer: consuming items from the model bounded buffer until there are no elements left to consume.
2. Partial reducing: reducing the consumed items into a submap. This is partial reduction as only a subset of the Key-Value values with the same Key are combined by a thread
3. Reduction Splitting: Splitting the elements of its partial reduction among the reduction queues of different slaves.
4. Final reduction: Processing its reduction queue to create the final values of the keys it has been assigned.

Here are the detailed steps:

The loop begins each iteration by invoking the take() operation on the model bounded buffer, which blocks if the buffer is empty. As we saw above, a consumed Key-Value item may be a regular (Token, Count) value or an end of input (Null, Null) value.

Processing of regular items: A regular item is stored in a slave list of Key-Value objects – the type of this list is your choice.

Processing of end of input items: An end of input item triggers the following steps:

1. Partial reduction: The slave accesses the slave list to compute a partially reduced slave Map by passing this list to the reduce() method (of the reducer returned by the getter of the reducer factory). You will use this method again to create a final result submap, this time passing it a different Key-Value list.
2. Splitting/partitioning of partially reduced map: Next the slave distributes the (key, value) pairs among the reduction queues stored in the model. To do so, it calls the getPartition() method (in the partitioner returned by the getter of the partitioner factory) to determine the reduction queue of each element of the partially reduced map. It retrieves each element of the partially reduces submap, and calls the partition() method with the key and value of the element, and the number of slave threads - which is the number of partitions. The return value of the method number list gives the index of the list in the reduction-queue list into which this element will be appended. After all slaves have completed this step, the elements

in their partially reduced maps will be distributed in the reduction queues in the reduction-queue list.

3. The slave now executes the barrier() method of the barrier in the model to wait for other slaves to complete their splitting/partitioning step.
4. Full Reduction: Now the slave takes a different role - full reduction of its reduction queue, which is the queue in the reduction-queue list whose index is the thread number of the slave. To do so, it calls the reduce() method it invoked in step 1, this time passing to the method its reduction queue rather than the items it consumed from the bounded buffer.
5. After completing the reduction the slave updates it's reduction-queue list to contain the reduced values
6. *Synchronized wait*: The slave now calls a synchronized method in its runnable that does a wait on itself. (Why do this wait rather than calling the blocking take() immediately?) The slave notifying method will unblock it.

The following trace excerpt illustrates this behavior:

// The master creates slaves (see above)

// Two of the created threads are scheduled, request dequeue and block on empty buffer

```
Thread[Slave0,5,main]:Slave:dequeue:[]
Thread[Slave1,5,main]:Slave:dequeue:[]
```

// The master enqueues two items without unblocking (see above)

…

// Slave1's dequeue request returns with the second enqueued item

```
Thread[Slave1,5,main]:Slave:dequeue:(orange,1)
```

// Slave2's dequeue request returns with the first enqueued item

```
Thread[Slave0,5,main]:Slave:dequeue:(apple,1)
```

// Why were the two slave's dequeue requests printed in an order that is opposite of the one in which the items were dequeued

// Slaves and master block and unblock as items are produced/consumed

…

// Slave2 dequeues the first end of input token, which triggers its processing of its sublist of dequeued items

```
Thread[Slave2,5,main]:Slave:dequeue:(null,null)
```

// Slave0 dequeues the second end of input token to start processing its sublist

```
Thread[Slave0,5,main]:Slave:dequeue:(null,null)
```

// Slave1 dequeues the third end of input token to start processing its list

```
Thread[Slave1,5,main]:Slave:dequeue:(null,null)
```

// The slaves do partial reduction of their sublists using the reducer

```
Thread[Slave2,5,main]:mapreduce.ATokenCountingReducer@79774f33:reduce:[(orange
,1), (lychee,1), (orange,1)]:{orange=2, lychee=1}
Thread[Slave0,5,main]:mapreduce.ATokenCountingReducer@79774f33:reduce:[(orange
,1), (mango,1), (soursop,1), (mango,1)]:{orange=1, soursop=1, mango=2}
Thread[Slave1,5,main]:mapreduce.ATokenCountingReducer@79774f33:reduce:[(apple,
1), (lychee,1), (lychee,1), (lychee,1)]:{apple=1, lychee=3}
```

// The slaves assign their partially computed key-value pairs to partitions and then wait on the barrier

```
Thread[Slave0,5,main]:Partitioner:Partition Assigned:lychee:2:1:3
Thread[Slave0,5,main]:Partitioner:Partition Assigned:mango:1:1:3
Thread[Slave2,5,main]:Partitioner:Partition Assigned:orange:2:1:3
Thread[Slave0,5,main]:Barrier:Barrier Wait Start:Barrier:3:1
Thread[Slave1,5,main]:Partitioner:Partition Assigned:lychee:1:1:3
Thread[Slave2,5,main]:Partitioner:Partition Assigned:lychee:1:1:3
Thread[Slave1,5,main]:Partitioner:Partition Assigned:soursop:1:2:3
```

// The slaves unblock from the barrier and print their partitions

```
Thread[Slave0,5,main]:mapreduce.ABarrier@436db750:Barrier Release
All:mapreduce.ABarrier@436db750:3:3
Thread[Slave1,5,main]:mapreduce.ABarrier@436db750:Barrier Wait
End:mapreduce.ABarrier@436db750:3:0
Thread[Slave0,5,main]:mapreduce.ATokenCountingSlave@57537eb9:Splits After
Barrier:0:[(apple,1)]
..
```

//The slaves do the final reduction of their partitions and signal finishing of the joined task

```
Thread[Slave0,5,main]:mapreduce.ATokenCountingReducer@79774f33:reduce:[(apple,
1)]:{apple=1}
Thread[Slave0,5,main]:mapreduce.AJoiner@16aa0a0a:Joiner
Finished:mapreduce.AJoiner@16aa0a0a:3:1
...
```

// Each slave now waits on its runnable for the next round of input

```
Thread[Slave2,5,main]:Slave:wait
```

…

// The next round start for slaves

```
Thread[Slave1,5,main]:Slave:dequeue:[(cherry,1), (blueberry,1)]
Thread[Slave1,5,main]:Slave:dequeue:(cherry,1)…
```

Terminating the Program

An easy way to terminate the program is to execute System.exit() in the main thread. In general, this is not a safe operation, as the other threads may need to perform some cleanup before terminating. You will gain experience with a more robust approach involving the concept of interrupting a thread. In this approach, you must ensure that when the user types “quit”, the main and slave threads break out of the controller and run loops, respectively.

The master thread, through the controller, is the one that receives the quit command. It must not only break out of its loop but also signal to each slave thread to break out of its thread. It can do so by calling interrupt() on each of the slave threads. (Why not instead call the slave notifying method on the Runnable of each thread?) Defined a new method called *terminate*() in the model that is called by the controller, which interrupts the slaves.

An interrupt() of a thread that is blocked on some operation causes the operation to unblock with an InterruptedException. This means control transfers to a catch block that can handle this exception. By putting this catch block in or outside the run loop, you can make the slave thread break out of the loop to terminate itself.

The slave runnable should call this method before terminating:

```
/**
 * To be called in the controller (A1) and slave runnable (A2)
 */
protected void traceQuit() {
      trace (QUITTING);
}
```

A slave may be blocked either because it has executed a take() on an empty bounder buffer, or because it has executed a wait() on its Runnable; so an InterruptedException may be thrown in two different places in your code.

Here are trace excerpts for the two conditions.

Blocked on wait on Runnable:

```
// After the second round completes, the master thread prompts for a new line
Thread[main,5,main]:Controller:Please enter quit or a line of tokens separated
by spaces
Quit

// The controller calls a method in the model that calls interrupt() on each thread waiting
on its runnable, which unblocks the threads and transfers control to the exception
handler for the wait. The exception handler prints the stack trace and breaks from the
runnable loop
java.lang.InterruptedException
Thread[Slave1,5,main]:Slave:Quitting
Thread[Slave0,5,main]:Slave:Quitting
Thread[Slave2,5,main]:Slave:Quitting
      at java.lang.Object.wait(Native Method)
      at java.lang.Object.wait(Object.java:502)
      at mapreduce.AnAbstractSlave.synchronizedWait(AnAbstractSlave.java:35)
      at mapreduce.AnAbstractSlave.run(AnAbstractSlave.java:50)
      at java.lang.Thread.run(Thread.java:745)
java.lang.InterruptedException
      at java.lang.Object.wait(Native Method)
      at java.lang.Object.wait(Object.java:502)
      at mapreduce.AnAbstractSlave.synchronizedWait(AnAbstractSlave.java:35)
      at mapreduce.AnAbstractSlave.run(AnAbstractSlave.java:50)
      at java.lang.Thread.run(Thread.java:745)
java.lang.InterruptedException
      at java.lang.Object.wait(Native Method)
      at java.lang.Object.wait(Object.java:502)
      at mapreduce.AnAbstractSlave.synchronizedWait(AnAbstractSlave.java:35)
      at mapreduce.AnAbstractSlave.run(AnAbstractSlave.java:50)
      at java.lang.Thread.run(Thread.java:745)
// The controller indicates it is about to finish its loop
Thread[main,5,main]:Controller:Quitting
```

Blocked on blocking queue:

```
// The application is re-run
Thread[main,5,main]:Controller:Please enter the number of threads
2
Thread[Slave0,5,main]:mapreduce.ATokenCountingSlave@58b40b80:dequeue:[]
Thread[main,5,main]:View:java.beans.PropertyChangeEvent[propertyName=Threads;
oldValue=null; newValue=[Thread[Slave0,5,main], Thread[Slave1,5,main]];
propagationId=null; source=Model]
Thread[main,5,main]:Controller:Please enter quit or a line of tokens separated
by spaces
Thread[Slave1,5,main]:mapreduce.ATokenCountingSlave@27155cbc:dequeue:[]
// No rounds of input, this time we quit immediately, causing again an interrupt to be sent
to each slave
Quit
// This time the slave's are blocked not on waits on their runnables, but on the blocking
queue, again the exception handler quits the runnable loop
java.lang.InterruptedException
Thread[Slave0,5,main]:mapreduce.ATokenCountingSlave@58b40b80:Quitting
Thread[Slave1,5,main]:mapreduce.ATokenCountingSlave@27155cbc:Quitting
        at
java.base/java.util.concurrent.locks.AbstractQueuedSynchronizer$ConditionObjec
t.reportInterruptAfterWait(AbstractQueuedSynchronizer.java:2050)
        at
java.base/java.util.concurrent.locks.AbstractQueuedSynchronizer$ConditionObjec
t.await(AbstractQueuedSynchronizer.java:2084)
        at
java.base/java.util.concurrent.ArrayBlockingQueue.take(ArrayBlockingQueue.java
:417)
        at mapreduce.AnAbstractSlave.run(AnAbstractSlave.java:43)
        at java.base/java.lang.Thread.run(Thread.java:844)
java.lang.InterruptedException
        at
java.base/java.util.concurrent.locks.AbstractQueuedSynchronizer$ConditionObjec
t.reportInterruptAfterWait(AbstractQueuedSynchronizer.java:2050)
        at
java.base/java.util.concurrent.locks.AbstractQueuedSynchronizer$ConditionObjec
t.await(AbstractQueuedSynchronizer.java:2084)
        at
java.base/java.util.concurrent.ArrayBlockingQueue.take(ArrayBlockingQueue.java
:417)
        at mapreduce.AnAbstractSlave.run(AnAbstractSlave.java:43)
        at java.base/java.lang.Thread.run(Thread.java:844)
// The controller indicates it is about to finish its loop
Thread[main,5,main]:Controller:Quitting
```

General-Structure of Single-Process Map-Reduce

Let us summarize the key elements of our algorithm above.

1. *Mapping*: The master process breaks up a sequence of input tokens into a list of Key-Value objects.
2. *Split*: The list is divided among slave threads, whose number is determined by the user.
3. *Partial reduction and split*: Each slave runnable processes its sub-list and splits the result of this processing in appropriate reduction queues
4. *Final reduction*: The slaves now process the reduction queues to compute different parts of the result.
5. *Final result*: The master thread prints the result once its join method returns.

This discussion is independent of the token counting algorithm implemented above as it leaves details of the mapping and reduction steps unspecified. It describes the general, problem-independent, structure of a *single-process* map-reduce implementation of a parallelizable problem.

Parallel Summation

The output above assumed the word-counting map class was used by the main program. By changing the mapper in the mapper factory, as in A1, we can create a different, integer summing main program,

The following excerpt from the trace illustrates number counting:

```
Thread[main,5,main]:Controller:Please enter the number of threads
3
…
Thread[main,5,main]:Controller:Please enter quit or a line of tokens to be
processed separated by spaces
// This time the entered tokens are numbers
       1 2 3 4
…
// The enqueued and dequeued items all the same key
// The value of the Nth item is the Nth number entered
Thread[main,5,main]:Model:enqueue:(Sum,1)
…
Thread[Slave1,5,main]:Slave:dequeue:(Sum,1)
…
Thread[main,5,main]:Model:enqueue:(Sum,2)

…
// The partial reductions are done in parallel
Thread[Slave0,5,main]:Slave:reduce:[(Sum,2)]:{Sum=2}
Thread[Slave1,5,main]:Slave:reduce:[(Sum,1), (Sum,4)]:{Sum=5}
Thread[Slave2,5,main]:Slave:reduce:[(Sum,3)]:{Sum=3}
// All partial results are assigned to the same partition
Thread[Slave0,5,main]:Partitioner:Partition Assigned:lychee:2:1:3
Thread[Slave0,5,main]:Partitioner:Partition Assigned:mango:1:1:3
Thread[Slave2,5,main]:Partitioner:Partition Assigned:orange:2:1:3
Thread[Slave0,5,main]:Barrier:Barrier Wait Start:Barrier:3:1
Thread[Slave1,5,main]:Partitioner:Partition Assigned:lychee:1:1:3
Thread[Slave2,5,main]:Partitioner:Partition Assigned:lychee:1:1:3
```

```
Thread[Slave1,5,main]:Partitioner:Partition Assigned:soursop:1:2:3
```

// Each slave adds the value parts of its Key-Value items
// The result is sum of all of the values totaled by the slave runnables
```
Thread[Slave1,5,main]:View:java.beans.PropertyChangeEvent[propertyName=Result;
oldValue=null; newValue={Sum=10}; propagationId=null; source=Model]
```
…

Submission Instructions

Same as in A0 and A1.

Gotchas (Possible Problems and Solutions)

Ideally, all or some of the difficulties you submitted could be added here.

1. When using traceAddedToMap(originalMap, reducedMap), make sure to call the trace after adding the reducedMap to the originalMap. That is, originalMap should contain the elements of reducedMap.
2. In order to call the slave notifying method on the slave runnable , you must keep a list of references to the Slave objects (runnables).
3. If you're getting an illegalMonitorException on the this.wait() call in your run method, synchronize your run method.
4. If you're getting an illegalMonitorException in general, check to make sure that all methods that call wait() and notify() are synchronized. It can help to create methods such as makeWait() and makeNotify() that make wait() and notify() calls and are synchronized, then call these from the calling method when needing to wait() or notify(), so that the entire calling method doesn't have to be synchronized - make sure you own the lock for the object you are trying to wait or notify on. So, in the model, you can't do "synchronized(this) {myRunnable.notify();}", you should do "synchronized(myRunnable) {myRunnable.notify();}"
5. The formula for partition size, s = Math.ceil(('z' - 'a' + 1) / p), has p as an int. This has the unintended result of rounding (('z' - 'a' + 1) / p) down before Math.ceil() can round the value up. The fix is to cast p to a double before you divide by p.
6. Only prompt the user for the number of threads once (at the very beginning). All subsequent inputs will be mapped and reduced using the number of threads specified at the very start.
7. Make sure the old value and new value in fired property change events are not equal, otherwise the event does not get broadcast. You can make the old value always null to ensure that.
8. Make sure you update the class objects returned in the following methods of the config file: public Class getStandAloneIntegerSummer(); public Class getStandAloneTokenCounter()